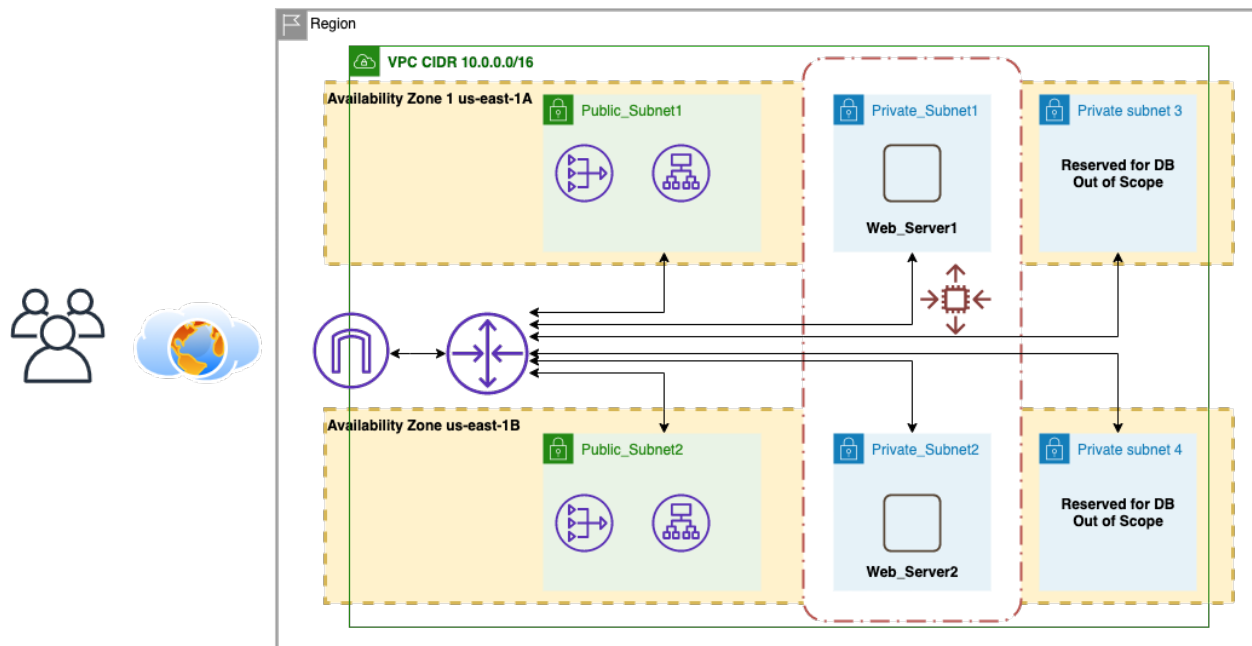


Terraform & AWS

Using Terraform to Deploy a custom VPC for a Multi-Tier Web Application hosted in AWS



Introduction:

- The project will use Terraform and HCL script to deploy a highly available, auto-scaled, web application using EC2 instances in AWS.
- The project will use Application load balancing to distribute the load across two availability zones in AWS in North Virginia (us-east-1) region.
- Database creation is out of scope.

Required Project Resources :

- A custom VPC in AWS
- 4 subnets, 2 public and 2 private subnets in two different availability zones (us-east-1a and us-east-1b).
- An Internet Gateway to be created and attached to the VPC, use IGW as the name.
- Two NAT Gateways (NATGW1 and NATGW2) to be created in the public subnets, one each.
- 2 security groups WebSG (allows SSH, HTTP inbound and all traffic outbound) and ALBSG (allows HTTP inbound and all traffic outbound)

- 2 EBS backed EC2 instances (Instance A and Instance B)
- A target group, WebTG
- An application load balancer WebALB
- A launch template and an auto scaling group

Things to think about:

- Do you need one or more route tables? Why?
- How will the instances of the private subnet(s) connect to the internet and AWS services?
- Can you place the Web/App tier instances in private subnets 1 and 2?

General Guidelines:

- Avoid using static assignments.
- Leverage variables all the way.
- Use a variables.tf file for the variable declarations.
- User a terraform.tfvars file for the variables static value assignments.

Required Steps:

Design and deploy the solution for a multi-tier web application in a custom AWS VPC.

Task 1 : Create the VPC and the Subnets:

- Create a **custom VPC** , custom_VPC, with CIDR block 10.0.0.0/16. The VPC should enable DNS hostnames for EC2 instances created in the VPC.
- Create an Internet Gateway and attach it to the VPC, use the name tag IGW-custom_VPC.
- Create two Public subnets with the names Public_Subnet1 and Public_Subnet2 in two different Availability Zones, US-east-1a and us-east-1B in US-east-1 region (you may use a different region but stick to the 1a and 1b in the choice of availability zones).
 - Use 10.0.10.0/24 and 10.0.20.0/24 ranges for these two subnets.
 - The subnets must allow assigning public IP addresses to the instances created
 - You may use a different region but stick to the 1a and 1b in the choice of availability zones.
- Create a route table for these subnets. The route table must allow access to the internet. Use the name Public_RT.
- Create two Private subnets in the same AZs as above (1a and 1b) with the names, Private_Subnet1 and Private_Subnet2.
 - Use 10.0.100.0/24 and 10.0.200.0/24 for the subnets.

Task 2 : Create Two NAT Gateways

- Create two elastic IP addresses, EIP1 and EIP2
- Create two public NAT Gateways (NATGW1 and NATGW2) in availability zones (AZ) us-east-1a and another in us-east-1b.
 - They need to be Public NAT Gateways (created in public subnets).
 - Create NATGW1 in Public_Subnet1 and NATGW2 in Public_Subnet2
 - Assign the Elastic IP addresses to each NAT gateway such that EIP1 is assigned to NATGW1 and EIP2 is assigned to NATGW2.
 - Create two separate route tables for the private subnets with the name Private_RT_1 and Private_RT_2.
 - Add a route in each private route table for 0.0.0.0/0 assign the target as follows:
 - Private_RT_1 will use the NATGW1 and Private_RT_2 will use the NATGW2.

Task 3 : Create Two Security Groups

- Create 2 security groups WebSG (allows HTTP inbound and all traffic outbound) and ALBSG (allows HTTP inbound and all traffic outbound)

Task 4 : Create an instance profile that will use an IAM Role to allow the instances to connect to SSM:

- Use the name LT_Profile for the instance profile name.
- The IAM role to be created must have the permissions required to connect to SSM (Policy : AmazonEC2RoleforSSM).
- Use EC2_SSM as the name of the role

Task 5 : Create High Availability using an Application Load Balancer:

- Create a target group with the name WebTG.
- The target group will use the port 80 (HTTP) for traffic forwarding and health checks.
- Launch an **internet-facing** application load balancer with the name WebALB that will load balance to these two instances using HTTP.
 - The application load balancer must be enabled in the two public subnets you have created above.
- Use the security group with the name ALBSG that will allow port 80 (HTTP) on the inbound from any source and allows all traffic in the outbound to all destinations.

Task 6 : Create a launch template and an auto scaling group to be integrated with the ALB and take care of creating and terminating instances as required

- Create a launch template using WebSG as the security group, in the two private subnets inside the custom VPC.
- The instance type is t2.micro and Linux AMI (use AWS Linux 2023).
- The template must have the user data script :

```
#!/bin/bash
yum update -y
yum install httpd -y
systemctl start httpd
systemctl enable httpd
echo " This is an app server in AWS Region US-EAST-1 " > /var/www/html/index.html
```

- The template must use the created EC2 instance profile, LT_profile
- Create an auto scaling group with the name ASG with manual scaling that will ensure elasticity and cost effectiveness.
- The Auto Scaling group should be enabled in the custom VPC and on the two Private subnets above.
- Use desired instance equal 2, the minimum equal 1, and the maximum equal 4.
- Add the created load balancer above to the auto scaling group using the WebTG target group.
- Ensure that the ASG used ELB health checks

Task 7 : Output the DNS hostname of the Load balancer, and the two instance IDs as part of your configuration script.

Task 8 : review your configuration file and plan then deploy it.

Verification:

- Check the AWS console for two EC2 instances created by the auto scaling group, one in AZ 1a and one in AZ 1b.
- The auto scaling should monitor the initial two instances and be able to add instances on-demand and replace the failed instances when this happens.
- From the console, Once the auto scaling group adds the new instances to the WebTG, you should check under the WebTG to see if they turn healthy.
- Once they turn healthy, test again from the browser with the DNS hostname of the ALB.
- You should get the message "This is an app server in AWS Region US-EAST-1".
- If it works, congratulations on finishing this amazing project on AWS.
- If not, troubleshoot to see what has gone wrong and verify that:

- The instances are showing as healthy under the WebTG group.
- You can connect to the two EC2 instances using SSM from within the AWS management console.
- The EC2 instances have Apache installed.
- If not troubleshoot to see what may have went wrong.
 - If not, you need to use connect using SSM to each instance and ensure using the command “sudo systemctl status httpd” that Apache is active and running.
 - Check the security group on the instance is WebSG and make sure it allows port 80 (HTTP) in the inbound direction.
 - Make any necessary changes and test again.
 - Use the command “curl http://<instancepublicip> and see if it returns the message in the script above.
 - If not, run the script commands above manually one by one preceded by sudo keyword, test again from a browser.
- It should work now.
- Repeat the same test with the second instance.

Task 9 : Cleanup

- Once completed successfully, destroy the deployed infrastructure.

•

```
#!/bin/bash
yum update -y
yum install httpd -y
systemctl start httpd
systemctl enable httpd
sudo groupadd www
sudo usermod -a -G www ec2-user
sudo chgrp -R www /var/www
sudo chmod 2775 /var/www
find /var/www -type d -exec sudo chmod 2775 {} +
find /var/www -type f -exec sudo chmod 0664 {} +
echo " This is server *1* in AWS Region US-EAST-1 in AZ US-EAST-1A " > /var/www/html/index.html
```